Next time: resource accounting

> 下次课：资源核算

CS336: Language Models From Scratch (Spring 2026)

> CS336：从零构建语言模型（2026 春季）


Percy Liang
Instructor


Tatsunori Hashimoto
Instructor


Marcel Rød
CA


Herman Brunborg
CA


Steven Cao
CA

...bringing you the 3rd offering of CS336.

> .....为大家带来 CS336 的第三次开课。

Lectures from 2nd offering (Spring 2025) are on YouTube.

> 第 2 次（2025 春季）开课的录像在 YouTube 上。

What's new?

> 有哪些新内容?

- Same 'from scratch' philosophy

> 同样的"从零构建"理念

- Prioritize high value-per-time concepts, don't lose the forest for the trees

> 优先讲单位时间价值最高的概念，不要只见树木不见森林

- More coverage of modern LM ingredients (mixture of experts, long-context, agents)

> 更多覆盖现代 LM 的构成要素（混合专家、长上下文、智能体）

Why did we make this course?

> 我们为什么要开这门课?

Problem: researchers are becoming **disconnected** from the underlying technology.

> 问题：研究者正在与底层技术脱节。

- 2016: researchers implemented and trained their own models.

> 2016：研究者自己实现并训练模型。

- 2018: researchers downloaded models (e.g., BERT) and fine-tuned them.

> 2018：研究者下载模型（如 BERT）并微调。

- Today: researchers prompt API models (e.g., GPT/Claude/Gemini).

> 如今：研究者给 API 模型写 prompt（如 GPT/Claude/Gemini）。

Moving up levels of abstraction boosts productivity, but

> 提升抽象层级能提高生产力，但是

- These abstractions are leaky (in contrast to programming languages or operating systems).

> 这些抽象是漏的（与编程语言或操作系统不同）。

- There is still fundamental research to be done that requires tearing up the stack.

> 仍然存在需要掀翻整个技术栈去做的基础研究。

Full understanding of this technology is necessary for **fundamental research**.

> 要做基础研究，就必须对这门技术有完整的理解。

Philosophy of this course: **understanding via building**.

> 本课的理念：通过构建来理解。

But there's one small problem...

> 但有一个小问题……

The industrialization of language models

> 语言模型的工业化



Frontier models are really expensive:

> 前沿模型真的很贵：

- 2023: GPT-4 supposedly cost $100M to train. [article]

> 2023：据说 GPT-4 的训练花费高达 $100M。 [article]

- 2025: xAI builds cluster with 230K GPUs for training Grok. [article]

> 2025：xAI 为训练 Grok 建起拥有 230K 块 GPU 的集群。 [article]

There are no public details on how frontier models are built.

> 前沿模型的构建方式没有任何公开细节。

From the GPT-4 technical report [OpenAI+ 2023]:

> 摘自 GPT-4 技术报告 [OpenAI+ 2023]：

2 Scope and Limitations of this Technical Report

This report focuses on the capabilities, limitations, and safety properties of GPT-4. GPT-4 is a Transformer-style model [39] pre-trained to predict the next token in a document, using both publicly available data (such as internet data) and data licensed from third-party providers. The model was then fine-tuned using Reinforcement Learning from Human Feedback (RLHF) [40]. Given both the competitive landscape and the safety implications of large-scale models like GPT-4, this report contains no further details about the architecture (including model size), hardware, training compute, dataset construction, training method, or similar.

We are committed to independent auditing of our technologies, and shared some initial steps and ideas in this area in the system card accompanying this release.[2] We plan to make further technical details available to additional third parties who can advise us on how to weigh the competitive and safety considerations above against the scientific value of further transparency.

Frontier models are out of reach for us.

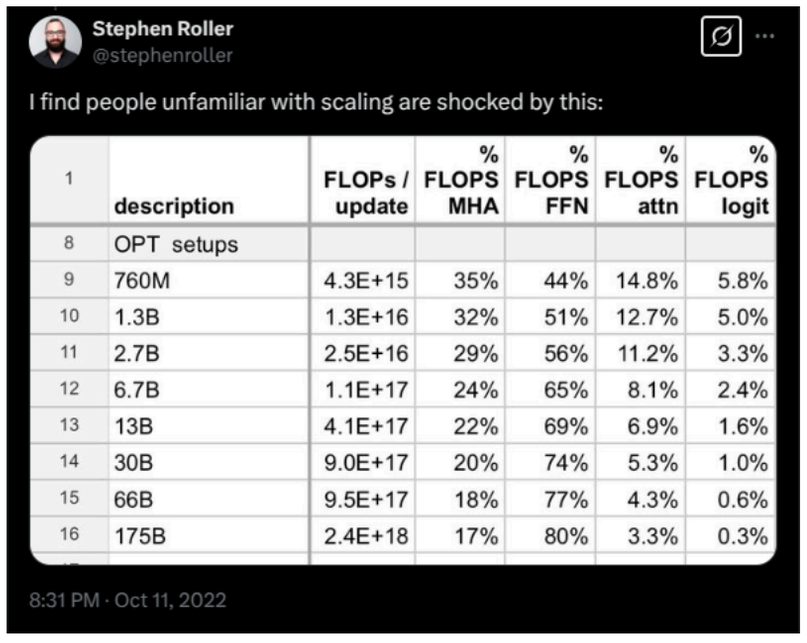
> 前沿模型对我们而言遥不可及。

We could build small language models (<1B parameters), but this might not be representative of large language models.

> 我们可以构建小语言模型（<1B 参数），但这可能不足以代表大语言模型。

Example 1: fraction of FLOPs spent in attention versus MLP changes with scale. [post]

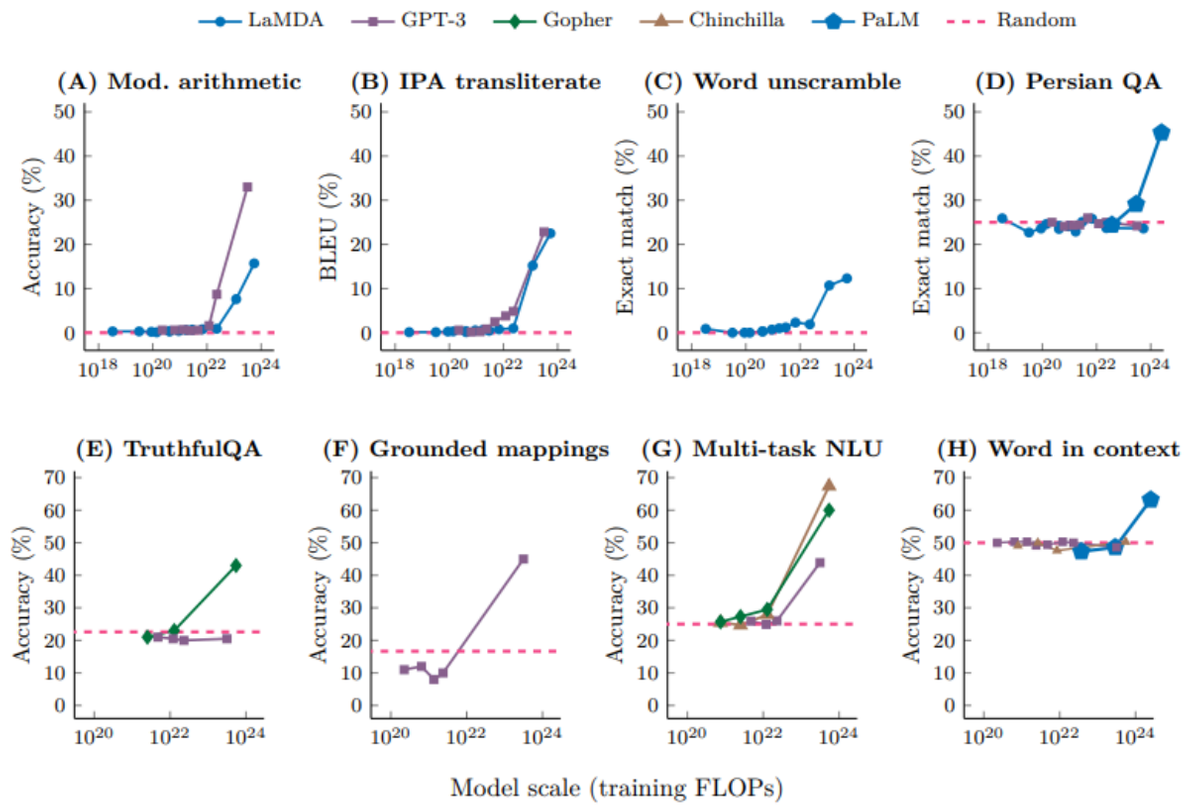
> 例 1：注意力与 MLP 所耗 FLOPs 的占比随规模而变化。 [post]



1	description	FLOPs / update	% FLOPS MHA	% FLOPS FFN	% FLOPS attn	% FLOPS logit
8	OPT setups					
9	760M	4.3E+15	35%	44%	14.8%	5.8%
10	1.3B	1.3E+16	32%	51%	12.7%	5.0%
11	2.7B	2.5E+16	29%	56%	11.2%	3.3%
12	6.7B	1.1E+17	24%	65%	8.1%	2.4%
13	13B	4.1E+17	22%	69%	6.9%	1.6%
14	30B	9.0E+17	20%	74%	5.3%	1.0%
15	66B	9.5E+17	18%	77%	4.3%	0.6%
16	175B	2.4E+18	17%	80%	3.3%	0.3%

Example 2: emergence of behavior with scale [Wei+ 2022]

> 例 2：行为随规模涌现 [Wei+ 2022]



What can we learn in this class that transfers to frontier models?

> 在这门课上，我们能学到哪些可以迁移到前沿模型的东西?

There are three types of knowledge:

> 知识有三类：

- **Mechanics**: how things work (what a Transformer is, how model parallelism works)

> 机制：事物如何运作（Transformer 是什么、模型并行如何工作）

- **Mindset**: squeezing the most out of the hardware, taking scaling seriously

> 心法：把硬件榨到极致、认真对待 scaling

- **Intuitions**: which data and modeling decisions yield good accuracy

> 直觉：哪些数据与建模决策能带来好精度

We can teach mechanics and mindset (these do transfer).

> 机制和心法我们可以教（这些确实可迁移）。

We can only partially teach intuitions (do not necessarily transfer across scales).

> 直觉我们只能教一部分（未必能跨规模迁移）。

Intuitions? 🤷

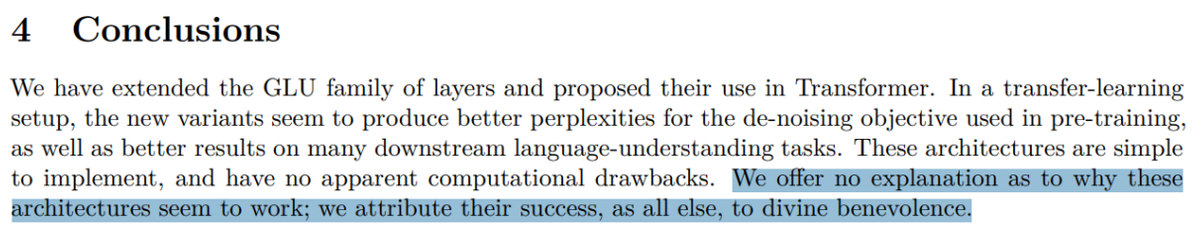
> 直觉呢? 🤷

Some design decisions are simply not (yet) justifiable and just come from experimentation.

> 有些设计决策（目前）无法给出论证，纯粹来自实验。

Example: Noam Shazeer paper that introduced SwiGLU [Shazeer 2020]

> 例：Noam Shazeer 提出 SwiGLU 的论文 [Shazeer 2020]



4 Conclusions

We have extended the GLU family of layers and proposed their use in Transformer. In a transfer-learning setup, the new variants seem to produce better perplexities for the de-noising objective used in pre-training, as well as better results on many downstream language-understanding tasks. These architectures are simple to implement, and have no apparent computational drawbacks. We offer no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence.

The bitter lesson

> 苦涩的教训

Wrong interpretation: scale is all that matters, algorithms don't matter.

> 错误解读：规模就是一切，算法不重要。

Right interpretation: algorithms that scale are what matter.

> 正确解读：真正重要的是能够随规模扩展的算法。

accuracy = efficiency x resources

In fact, efficiency is way more important at larger scales (can't afford to be wasteful).

> 事实上，规模越大，效率越重要（浪费不起）。

[Hernandez+ 2020] showed 44x algorithmic efficiency on ImageNet between 2012 and 2019.

> [Hernandez+ 2020] 的研究表明，2012 到 2019 年间 ImageNet 上的算法效率提升了 44 倍。

Framing: what is the best model one can build given a certain compute and data budget?

> 问题框架：在给定的算力和数据预算下，能构建出的最好模型是什么？

In other words, **maximize efficiency**!

> 换句话说，就是把效率最大化！

Pre-neural (before 2010s)

> 前神经时代（2010 年代之前）

- Language model to measure the entropy of English [Shannon 1950]

> 用语言模型测量英语的熵 [Shannon 1950]

- N-gram language models (used in machine translation and speech recognition systems) [Brants+ 2007]

> N-gram 语言模型（用于机器翻译和语音识别系统） [Brants+ 2007]

Neural ingredients (2010s)

> 神经时代组件（2010 年代）

- Long-Short Term Memory (LSTM) [Hochreiter+ 1997]

> 长短期记忆网络（LSTM） [Hochreiter+ 1997]

- First neural language model [Bengio+ 2003]

> 第一个神经语言模型 [Bengio+ 2003]

- Sequence-to-sequence modeling (for machine translation) [Sutskever+ 2014]

> 序列到序列建模（用于机器翻译） [Sutskever+ 2014]

- Adam optimizer [Kingma+ 2014]

> Adam 优化器 [Kingma+ 2014]

- Attention mechanism (for machine translation) [Bahdanau+ 2014]

> 注意力机制（用于机器翻译） [Bahdanau+ 2014]

- Transformer architecture (for machine translation) [Vaswani+ 2017]

> Transformer 架构（用于机器翻译） [Vaswani+ 2017]

- Mixture of experts [Shazeer+ 2017]

> 混合专家 [Shazeer+ 2017]

- Model parallelism [Huang+ 2018][Rajbhandari+ 2019][Shoeybi+ 2019]

> 模型并行 [Huang+ 2018][Rajbhandari+ 2019][Shoeybi+ 2019]

Early foundation models (late 2010s)

> 早期基础模型（2010 年代末）

- ELMo: pretraining with LSTMs, fine-tuning improves downstream tasks [Peters+ 2018]

> ELMo：用 LSTM 预训练，微调改进下游任务 [Peters+ 2018]

- BERT: pretraining with Transformer, fine-tuning improves downstream tasks [Devlin+ 2018]

> BERT：用 Transformer 预训练，微调改进下游任务 [Devlin+ 2018]

- Google's T5 (11B): cast everything as text-to-text [Raffel+ 2019]

> Google 的 T5（11B）：把一切都化作文本到文本 [Raffel+ 2019]

Embracing scaling

> 拥抱 scaling

- OpenAI's GPT-2 (1.5B): fluent text, first signs of zero-shot [Radford+ 2019]

> OpenAI 的 GPT-2（1.5B）：流畅文本，zero-shot 的最初迹象 [Radford+ 2019]

- Scaling laws: provide hope / predictability for scaling [Kaplan+ 2020]

> Scaling laws：为扩展带来希望/可预测性 [Kaplan+ 2020]

- OpenAI's GPT-3 (175B): in-context learning [Brown+ 2020]

> OpenAI 的 GPT-3（175B）：上下文学习 [Brown+ 2020]

- Google's PaLM (540B): massive scale, undertrained [Chowdhery+ 2022]

> Google 的 PaLM（540B）：规模巨大，训练不足 [Chowdhery+ 2022]

- DeepMind's Chinchilla (70B): compute-optimal scaling laws [Hoffmann+ 2022]

> DeepMind 的 Chinchilla（70B）：算力最优的 scaling laws [Hoffmann+ 2022]

Open models

> 开放模型

Early attempts (attempts to replicate GPT-3):

> 早期尝试（试图复刻 GPT-3）：

- EleutherAI's open datasets (The Pile) and models (GPT-J) [Gao+ 2020][Wang+ 2021]

> EleutherAI 的开放数据集（The Pile）与模型（GPT-J） [Gao+ 2020][Wang+ 2021]

- Meta's OPT (175B): GPT-3 replication, lots of hardware issues [Zhang+ 2022]

> Meta 的 OPT（175B）：GPT-3 复刻，一堆硬件问题 [Zhang+ 2022]

- Hugging Face / BigScience's BLOOM (176B): focused on data sourcing [Workshop+ 2022]

> Hugging Face / BigScience 的 BLOOM（176B）：专注数据获取 [Workshop+ 2022]

Credible open-weight models (weights + paper):

> 可信的开放权重模型（权重 + 论文）：

- Meta's Llama models [Touvron+ 2023][Touvron+ 2023][Grattafiori+ 2024]

> Meta 的 Llama 系列模型 [Touvron+ 2023][Touvron+ 2023][Grattafiori+ 2024]

- Mistral's models [Jiang+ 2023][Jiang+ 2024]

> Mistral 的模型 [Jiang+ 2023][Jiang+ 2024]

- DeepSeek's models [DeepSeek-AI+ 2024][DeepSeek-AI+ 2024][DeepSeek-AI+ 2024][DeepSeek-AI+ 2025][DeepSeek-AI+ 2025]

> DeepSeek 的模型 [DeepSeek-AI+ 2024][DeepSeek-AI+ 2024][DeepSeek-AI+ 2024][DeepSeek-AI+ 2025][DeepSeek-AI+ 2025]

- Alibaba's Qwen models [Qwen+ 2024][Yang+ 2025]

> 阿里巴巴的 Qwen 模型 [Qwen+ 2024][Yang+ 2025]

- Moonshot's Kimi models [Kimi Team 2025][Kimi Team 2026]

> 月之暗面的 Kimi 模型 [Kimi Team 2025][Kimi Team 2026]

- Z.ai's GLM models [GLM-4.5 Team 2025][GLM-5-Team 2026]

> Z.ai 的 GLM 模型 [GLM-4.5 Team 2025][GLM-5-Team 2026]

- Minimax's models [[MiniMax M2.5]]

> MiniMax 的模型 [[MiniMax M2.5]]

- Xiaomi's MIMO models [[Xiaomi MIMO v2]]

> 小米的 MIMO 模型 [[Xiaomi MIMO v2]]

These models are approaching closed models (GPT, Claude, Gemini, etc.).

> 这些模型正在逼近闭源模型（GPT、Claude、Gemini 等）。

Open-source models (weights + paper + code + data):

> 开源模型（权重 + 论文 + 代码 + 数据）：

- AI2's Olmo models [Groeneveld+ 2024][Team OLMo 2024][Team Olmo 2025]

> AI2 的 Olmo 模型 [Groeneveld+ 2024][Team OLMo 2024][Team Olmo 2025]

- NVIDIA's Nemotron models [Parmar+ 2024][NVIDIA+ 2025]

> NVIDIA 的 Nemotron 模型 [Parmar+ 2024][NVIDIA+ 2025]

- Marin's models (open development) [[Marin 8B retro]][[Marin 32B retro]]

> Marin 的模型（开放式研发） [[Marin 8B retro]][[Marin 32B retro]]

Openness is important for trust and innovation [Kapoor+ 2024]

> 开放性对信任与创新都很重要 [Kapoor+ 2024]

Ideas from open models enable us to teach CS336.

> 开放模型中的思想让我们得以开设 CS336。

What is a language model?

> 什么是语言模型?

- 2018 (BERT): something you fine-tune

> 2018（BERT）：拿来微调的东西

- 2020 (GPT-3): something you prompt

> 2020（GPT-3）：拿来写 prompt 的东西

- 2022 (ChatGPT): something you talk to [example conversation]

> 2022（ChatGPT）：与你对话的东西 [example conversation]

- 2026 (agents): something that acts autonomously [example trace]

> 2026（智能体）：自主行动的东西 [example trace]

The fundamentals are the same (attention, kernels, optimization).

> 基本原理是相同的（注意力、kernel、优化）。

The specs are different (longer context, inference efficiency matters even more).

> 规格要求不同（上下文更长、推理效率更加重要）。

This is an *executable lecture*, a program whose execution delivers the content of a lecture.

> 这是一份可执行讲义：一个通过运行来交付讲课内容的程序。

Executable lectures make it possible to:

> 可执行讲义使得以下成为可能：

- view and run code (since everything is code!),

> 查看并运行代码（因为一切都是代码！），

- see the hierarchical structure of the lecture

> 查看讲义的层级结构

All information online: [course website]

> 所有资料都在线上： [course website]

This is a 5-unit class.

> 这是一门 5 学分的课。

Comment from Spring 2024 course evaluation:

> 来自 2024 春季课程评价的留言：

> *The entire assignment was approximately the same amount of work as all 5 assignments from CS 224n plus the final project. And that's just the first homework assignment.*

> 整个作业的工作量大约相当于 CS 224n 全部 5 次作业加上期末项目的总和。而这还只是第一次作业。

Why you should take this course

> 为什么你应该选这门课

- You have an obsessive need to understand how things work.

> 你有一种执念，非要搞懂事物的运作原理不可。

- You want to build up your research engineering muscles.

> 你想练出研究工程的肌肉。

Why you should not take this course

> 为什么你不该选这门课

- You actually want to get research done this quarter. (Talk to your advisor.)

> 你这个学季真的想做出研究进展。（去和你的导师谈谈。）

- You are interested in learning about the hottest new techniques in AI (e.g., multimodality, RAG, etc.). (You should take a seminar class for that.)

> 你想学 AI 里最热门的新技术（如多模态、RAG 等）。（那你应该去上研讨课。）

- You want to get good results on your own application domain. (You should just prompt or fine-tune an existing model.)

> 你想在自己的应用领域拿到好结果。（那你直接给现有模型写 prompt 或微调即可。）

How you can follow along at home

> 如何在家跟学

- All lecture materials and assignments will be posted online, so feel free to follow on your own.

> 所有讲义和作业都会发布在网上，欢迎自行跟学。

- Lectures are recorded via CGOE.

> 课程通过 CGOE 录制。

Assignments

> 作业

- 5 assignments (basics, systems, scaling laws, data, alignment).

> 5 次作业（基础、系统、scaling laws、数据、对齐）。

- No scaffolding code, but we provide unit tests and adapter interfaces to help you check correctness.

> 没有脚手架代码，但我们提供单元测试和适配器接口帮助你检查正确性。

- Implement locally to test for correctness, then run on cluster for benchmarking (accuracy and speed).

> 先在本地实现并测试正确性，再到集群上跑基准（精度和速度）。

- Leaderboard for some assignments (minimize perplexity given training budget).

> 部分作业设有排行榜（在给定训练预算下最小化困惑度）。

AI policy

> AI 政策

- Coding agents can solve all the assignments, but you won't learn anything.

> 编程智能体能解决所有作业，但你什么也学不到。

- AI can be tremendously useful for answering questions and tutoring.

> AI 在答疑和辅导上能发挥极大作用。

- You must use our provided AGENTS.md file, which asks the AI to be pedagogically-minded.

> 你必须使用我们提供的 AGENTS.md 文件，它会让 AI 以教学为重。

- Please read our AI policy guide.

> 请阅读我们的 AI 政策指南。

Compute

> 算力

- Thanks to Modal for providing compute. 🙏

> 感谢 Modal 提供算力。🙏

- Please read the guide on how to access and use the compute.

> 请阅读关于如何访问和使用算力的指南。

Remember it's all about **efficiency**:

> 记住，一切都是效率问题：

- Resources: data + hardware (compute, memory, communication bandwidth)

> 资源：数据 + 硬件（算力、内存、通信带宽）

- How do you train the best model given a fixed set of resources?

> 在固定的资源下，如何训练出最好的模型？

Today, we are compute-constrained, so design decisions will reflect squeezing the most out of given hardware.

> 如今我们受限于算力，因此设计决策会围绕把给定硬件榨到极致展开。

- Systems: clearly about efficiency

> 系统：显然关乎效率

- Tokenization: working with raw bytes is elegant, but compute-inefficient with today's model architectures

> 分词：直接处理原始字节很优雅，但在如今的模型架构下计算效率不高

- Model architecture: many changes motivated by reducing memory or FLOPs (e.g., sharing KV caches, sliding window attention)

> 模型架构：许多改动是为了降低内存或 FLOPs（如共享 KV cache、滑动窗口注意力）

- Data filtering: avoid wasting precious compute updating on bad / irrelevant data

> 数据过滤：避免在糟糕/无关的数据上做更新，浪费宝贵的算力

- Scaling laws: use less compute on smaller models to do hyperparameter tuning

> Scaling laws：用更少的算力在小模型上做超参数调优

Tomorrow, we will become data-constrained...

> 未来，我们将变为数据受限……

Goal: be able to train a basic language model

> 目标：能够训练一个基础语言模型

Components: tokenization, model architecture, training

> 组成：分词、模型架构、训练

Tokenization

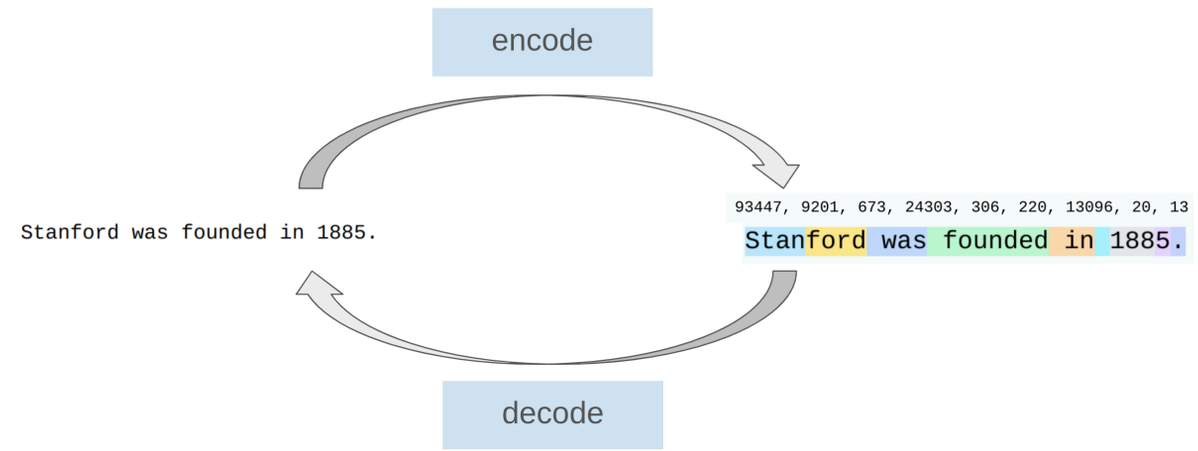
> 分词（Tokenization）

What are the atoms that the model operates on?

> 模型操作的基本单元是什么?

Formally: a tokenizer converts between raw inputs (bytes) and sequences of integers (tokens)

> 形式化地说：分词器在原始输入（字节）与整数序列（token）之间转换



Popular tokenizer: **Byte-Pair Encoding** (BPE) [Sennrich+ 2015]

> 流行的分词器：字节对编码（BPE） [Sennrich+ 2015]

Intuition: break input into frequently-occuring chunks

> 直觉：把输入切分成频繁出现的块

Efficiency lens

> 效率视角

- Reduce context length (1000 bytes → ~250 tokens)

> 缩短上下文长度（1000 字节 → 约 250 个 token）

- Adaptive computation (more modeling capacity on interesting parts of input)

> 自适应计算（把更多建模容量分配给输入中有趣的部分）

The dream: tokenizer-free model architectures, which operate directly on bytes [Xue+ 2021][Yu+ 2023][Pagnoni+ 2024][Deiseroth+ 2024][Hwang+ 2025]

> 终极梦想：无分词器的模型架构，直接在字节上运行 [Xue+ 2021][Yu+ 2023][Pagnoni+ 2024][Deiseroth+ 2024][Hwang+ 2025]

These are promising, but have not yet been scaled up to the frontier.

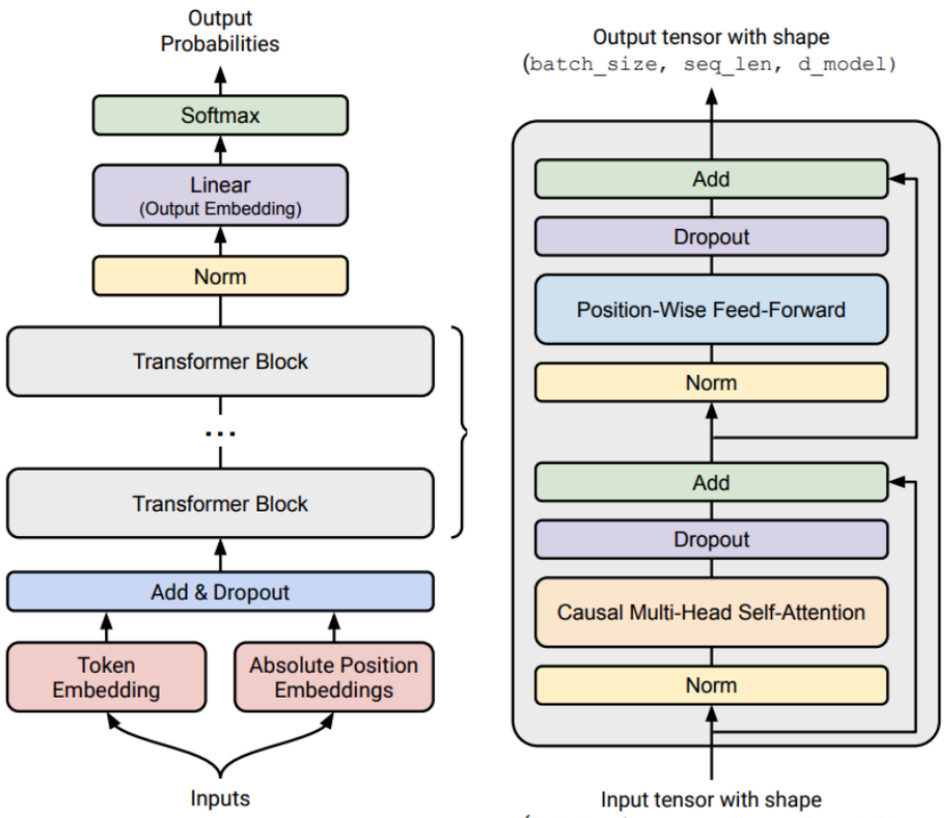
> 这些方向很有前景，但尚未扩展到前沿规模。

Model architecture

> 模型架构

Starting point: original Transformer [Vaswani+ 2017]

> 起点：原始 Transformer [Vaswani+ 2017]



Refinements:

> 改进项：

- Activation functions: ReLU, SwiGLU [Shazeer 2020]

> 激活函数：ReLU、SwiGLU [Shazeer 2020]

- Positional encodings: sinusoidal, RoPE [Su+ 2021]

> 位置编码：正弦、RoPE [Su+ 2021]

- Normalization: LayerNorm, RMSNorm, QK norm, pre-norm versus post-norm [Ba+ 2016][Zhang+ 2019][Dehghani+ 2023][Xiong+ 2020]

> 归一化：LayerNorm、RMSNorm、QK norm、pre-norm 与 post-norm [Ba+ 2016][Zhang+ 2019][Dehghani+ 2023][Xiong+ 2020]

- Attention: full, sparse/local attention, group-query attention (GQA), multi-head latent attention (MLA) [Child+ 2019][Ainslie+ 2023][DeepSeek-AI+ 2024]

> 注意力：全注意力、稀疏/局部注意力、分组查询注意力（GQA）、多头潜在注意力（MLA） [Child+ 2019][Ainslie+ 2023][DeepSeek-AI+ 2024]

- Recurrence/state-space models/linear attention: Mamba, Gated DeltaNet [Katharopoulos+ 2020][Dao+ 2024][Yang+ 2024][Lahoti+ 2026]

> 循环/状态空间模型/线性注意力：Mamba、Gated DeltaNet [Katharopoulos+ 2020][Dao+ 2024][Yang+ 2024][Lahoti+ 2026]

- MLP: dense, mixture of experts [Shazeer+ 2017][Fedus+ 2021]

> MLP：稠密、混合专家 [Shazeer+ 2017][Fedus+ 2021]

- Shape (hidden dimension, depth, number of heads, number of experts)

> 形状（隐藏维度、深度、头数、专家数）

Training

> 训练

How do you set the parameters of the model?

> 如何设置模型的参数?

- Loss function (e.g., multi-token prediction) [Gloeckle+ 2024][DeepSeek-AI+ 2024]

> 损失函数（如多 token 预测） [Gloeckle+ 2024][DeepSeek-AI+ 2024]

- Optimizer (e.g., AdamW, SOAP, Muon) [Kingma+ 2014][Loshchilov+ 2017][Vyas+ 2024][Keller 2024]

> 优化器（如 AdamW、SOAP、Muon） [Kingma+ 2014][Loshchilov+ 2017][Vyas+ 2024][Keller 2024]

- Initialization scale (e.g., Xavier init, muP) [Glorot+ 2010][Yang+ 2022]

> 初始化尺度（如 Xavier 初始化、muP） [Glorot+ 2010][Yang+ 2022]

- Learning rate schedule (e.g., cosine, WSD) [Loshchilov+ 2016][Hu+ 2024]

> 学习率调度（如 cosine、WSD） [Loshchilov+ 2016][Hu+ 2024]

- Regularization (e.g., dropout, weight decay)

> 正则化（如 dropout、权重衰减）

- Batch size (e.g., critical batch size) [McCandlish+ 2018]

> 批大小（如临界批大小） [McCandlish+ 2018]

- MoE specific: load balancing (e.g., aux-free) [Wang+ 2024][DeepSeek-AI+ 2024]

> MoE 专属：负载均衡（如 aux-free） [Wang+ 2024][DeepSeek-AI+ 2024]

Assignment 1 (basics)

> 作业 1（基础）

[GitHub][PDF]

- Implement BPE tokenizer

> 实现 BPE 分词器

- Implement Transformer, cross-entropy loss, AdamW optimizer, training loop

> 实现 Transformer、交叉熵损失、AdamW 优化器、训练循环

- Do resource accounting

> 做资源核算

- Train on TinyStories and OpenWebText

> 在 TinyStories 和 OpenWebText 上训练

- Leaderboard: minimize OpenWebText perplexity given 45 minutes on a B200 [last year's leaderboard]

> 排行榜：在 B200 上给定 45 分钟，最小化 OpenWebText 困惑度 [last year's leaderboard]

High-level principle: everything is about balancing the following:

> 高层原则：一切都在于平衡以下几点：

- Expressivity (can represent complex dependencies in the data)

> 表达力（能表示数据中的复杂依赖）

- Stability (keep parameter and gradient norms in goldilocks zone)

> 稳定性（让参数和梯度范数保持在恰到好处的黄金区间）

- Efficiency (runs fast on hardware, both training and inference)

> 效率（在硬件上跑得快，训练与推理皆然）

Goal: squeeze the most out of the hardware (GPU or TPU)

> 目标：把硬件（GPU 或 TPU）榨到极致

Components: kernels, parallelism, inference

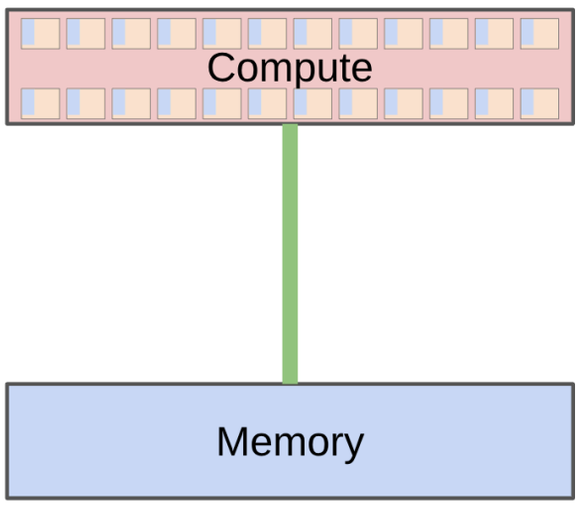
> 组成：kernel、并行、推理

Basics

> 基础

- Resource accounting: memory and compute characteristics of a model

> 资源核算：模型的内存与计算特性



- Model parameters must be moved from memory (HBM) to the compute (SMs)

> 模型参数必须从内存（HBM）搬到计算单元（SM）

- Example: B200 can perform 2.25 PFLOP/sec (bf16) with 8TB/sec memory bandwidth

> 例如：B200 能以 2.25 PFLOP/sec（bf16）进行计算，内存带宽为 8TB/sec

- Roofline analysis: understand whether we're compute-bound or memory-bound

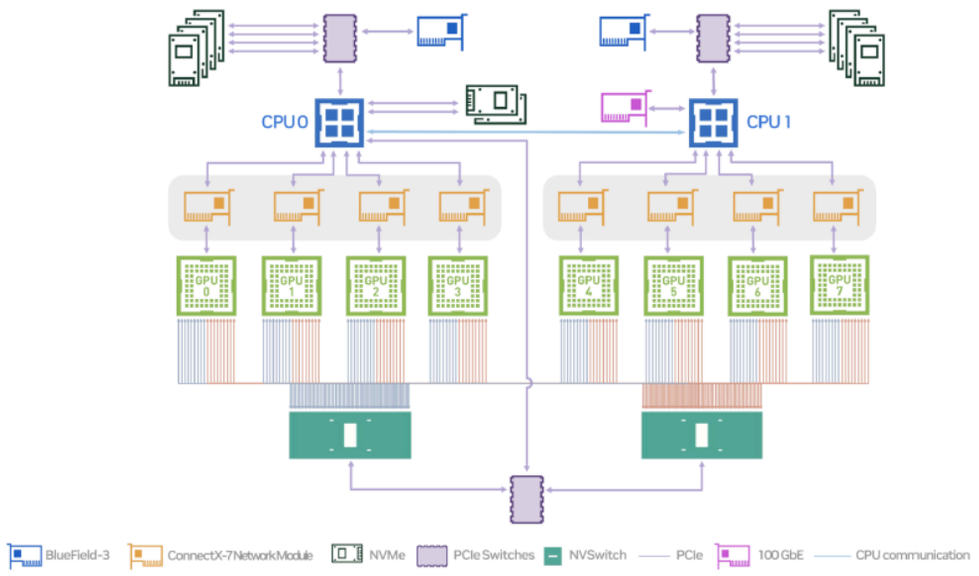
> Roofline 分析：判断我们受限于算力还是内存

- Benchmarking and profiling (nsight): see what happens in practice

> 基准测试与性能剖析（nsight）：看看实际发生了什么

DGX B200:

> DGX B200：



Kernels

> Kernels

- Kernel is a function that runs on GPU

> Kernel 是在 GPU 上运行的函数

- When using PyTorch, each primitive operation launches a standard kernel

> 使用 PyTorch 时，每个基本操作都会启动一个标准 kernel

- Can write custom kernels to make GPUs go brrr

> 可以编写自定义 kernel，让 GPU 狂飙

- Principle: organize computation to minimize data movement

> 原则：组织计算以最小化数据搬运

- Naive: read HBM; compute A; write HBM; read HBM; compute B; write HBM

> 朴素做法：读 HBM；计算 A；写 HBM；读 HBM；计算 B；写 HBM

- Fused: read HBM; compute A and B; write HBM

> 融合做法：读 HBM；计算 A 和 B；写 HBM

- Strategies: operator fusion (matmul + activation), tiling (FlashAttention)

> 策略：算子融合（matmul + 激活）、分块（FlashAttention）

- Warp divergence, memory coalescing, bank conflicts, occupancy, bulk-async memory transfers

> Warp 分歧、内存合并、bank 冲突、占用率、批量异步内存传输

- Write kernels in CUDA/**Triton**/CUTLASS/ThunderKittens

> 用 CUDA/Triton/CUTLASS/ThunderKittens 编写 kernel

Parallelism

> 并行

- What if we have 1024 GPUs?

> 如果我们有 1024 块 GPU 呢?

- Data movement between GPUs is even slower, but same 'minimize data movement' principle holds

> GPU 之间的数据搬运更慢，但"最小化数据搬运"的原则同样适用

- Use classic collective operations (e.g., gather, reduce, all-reduce)

> 使用经典的集合通信操作（如 gather、reduce、all-reduce）

- Shard memory (parameters, activations, gradients, optimizer states) across GPUs

> 把内存（参数、激活、梯度、优化器状态）分片到各 GPU

- How to split computation: {data,tensor,pipeline,sequence,expert} parallelism

> 如何切分计算：{数据、张量、流水线、序列、专家}并行

Inference

> 推理

Goal: generate tokens given a prompt (needed to actually use models!)

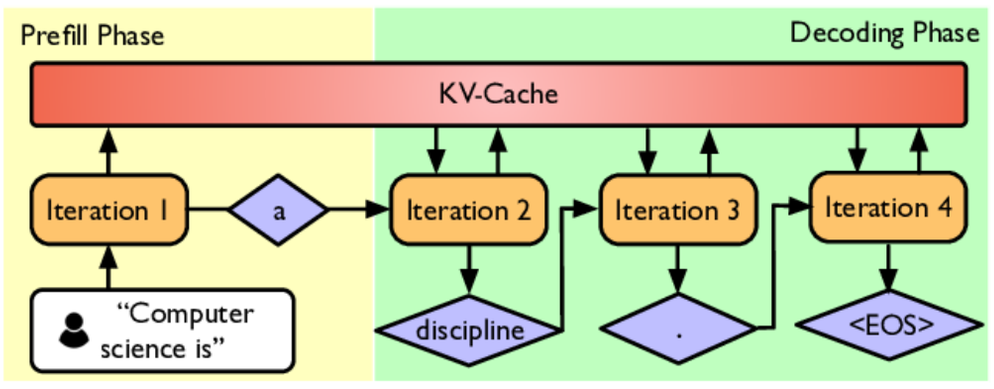
> 目标：给定 prompt 生成 token（实际使用模型所必需！）

Inference is also needed for reinforcement learning, test-time compute, evaluation

> 强化学习、测试时计算、评测同样需要推理

Two phases: prefill and decode

> 两个阶段：预填充与解码



- Prefill (similar to training): tokens are given, can process all at once (compute-bound)

> 预填充（与训练类似）：token 已知，可以一次性处理（受限于算力）

- Decode: need to generate one token at a time (memory-bound)

> 解码：需要一次生成一个 token（受限于内存）

Methods to speed up decoding:

> 加速解码的方法：

- Use cheaper model (via model pruning, quantization, distillation)

> 使用更省的模型（通过模型剪枝、量化、蒸馏）

- Speculative decoding: use a cheaper "draft" model to generate multiple tokens, then use the full model to score in parallel (exact decoding!)

> 投机解码：用更便宜的"草稿"模型生成多个 token，再用完整模型并行打分（精确解码！）

- Systems optimizations: fused kernels, continuous batching

> 系统层优化：融合 kernel、连续批处理

Assignment 2 (systems)

> 作业 2（系统）

[GitHub][PDF from Spring 2025]

- Implement a fused RMSNorm kernel in Triton

> 用 Triton 实现融合的 RMSNorm kernel

- Implement distributed data parallel training

> 实现分布式数据并行训练

- Implement optimizer state sharding

> 实现优化器状态分片

- Benchmark and profile the implementations

> 对实现做基准测试与性能剖析

Recommended book: How to Scale Your Model

> 推荐书目：How to Scale Your Model

- Nicely lays out how to approach systems for LLMs conceptually

> 把 LLM 系统的做法在概念层面讲得很清楚

- From Google, so it foregrounds TPUs, but high-level concepts are similar

> 出自 Google，因此以 TPU 为主，但高层概念是相通的

Setting: if you had 1e25 FLOPs of compute, what hyperparameters would you use to train a good model?

> 情境设定：如果你有 1e25 FLOPs 的算力，你会用什么超参数来训练一个好模型？

Too expensive to do hyperparameter tuning at full scale!

> 在全规模上做超参数调优太贵了！

Key conceptual shift: instead of a single scale, think of a **scaling recipe** (FLOPs → hyperparameters)

> 关键的概念转变：不要只想着单一规模，而要想一套扩展配方（FLOPs → 超参数）

For a scaling recipe:

> 对一套扩展配方：

- Run experiments to compute the loss at various smaller scales (e.g., up to 1e24 FLOPs)

> 在多个更小的规模上跑实验，计算损失（例如最高到 1e24 FLOPs）

- Fit a scaling law to predict the loss of the scaling recipe at the target scale (e.g., 1e25 FLOPs)

> 拟合 scaling law，预测该扩展配方在目标规模（例如 1e25 FLOPs）的损失

Now you can:

> 现在你可以：

1. Optimize the scaling recipe targeting a larger scale using smaller scale experiments

> 用小规模实验优化面向更大规模的扩展配方

2. Predict the loss at the target scale before actually running the experiment!

> 在真正跑实验之前预测目标规模的损失！

Scaling laws don't happen automatically, they require careful construction of a scaling recipe.

> Scaling laws 不会自动成立，它们需要精心构建扩展配方。

Parameterize the model in a way to get **hyperparameter transfer** [Yang+ 2022]

> 以能获得超参数迁移的方式参数化模型 [Yang+ 2022]

Predictability is at least as important as optimality!

> 可预测性至少与最优性同等重要！

Question: given a FLOPs budget (C = 6 N D), use a bigger model (N) or train on more tokens (D)?

> 问题：给定 FLOPs 预算（C = 6 N D），是用更大的模型（N），还是在更多的 token（D）上训练?

Classic compute-optimal scaling laws: [Kaplan+ 2020][Hoffmann+ 2022]

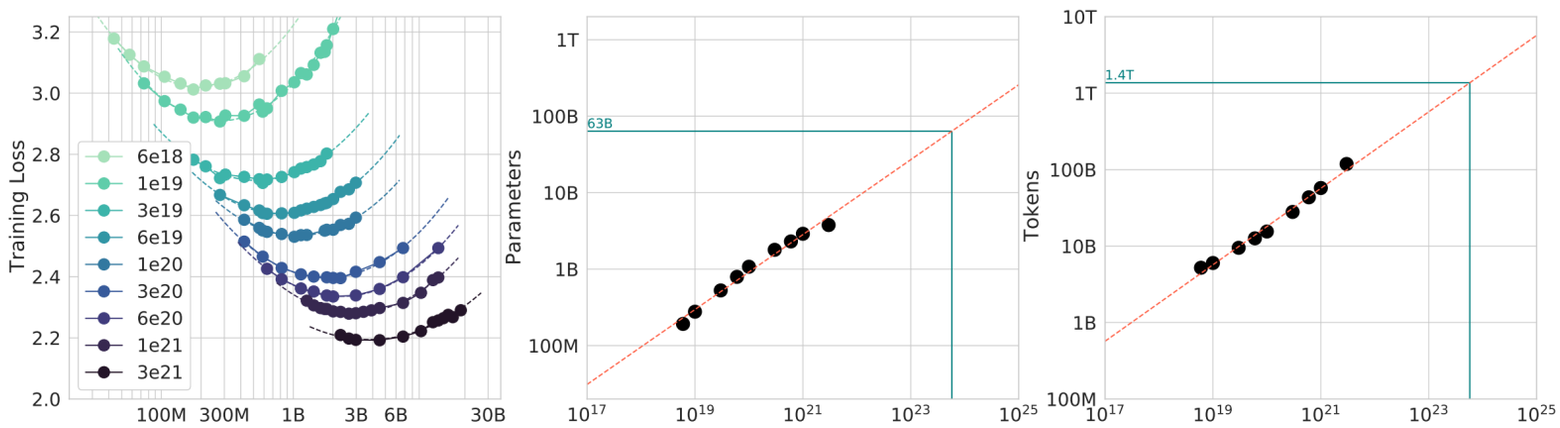
> 经典的算力最优 scaling laws： [Kaplan+ 2020][Hoffmann+ 2022]

- ISOFLOP curves: for multiple small FLOPs budgets, find optimal N

> ISOFLOP 曲线：对多个较小的 FLOPs 预算，找到最优的 N

- Then fit a scaling law to extrapolate to large FLOPs budgets

> 再拟合 scaling law，外推到大的 FLOPs 预算



TL;DR: D = 20 N is roughly optimal (e.g., 70B parameter model should be trained on ~1.4T tokens)

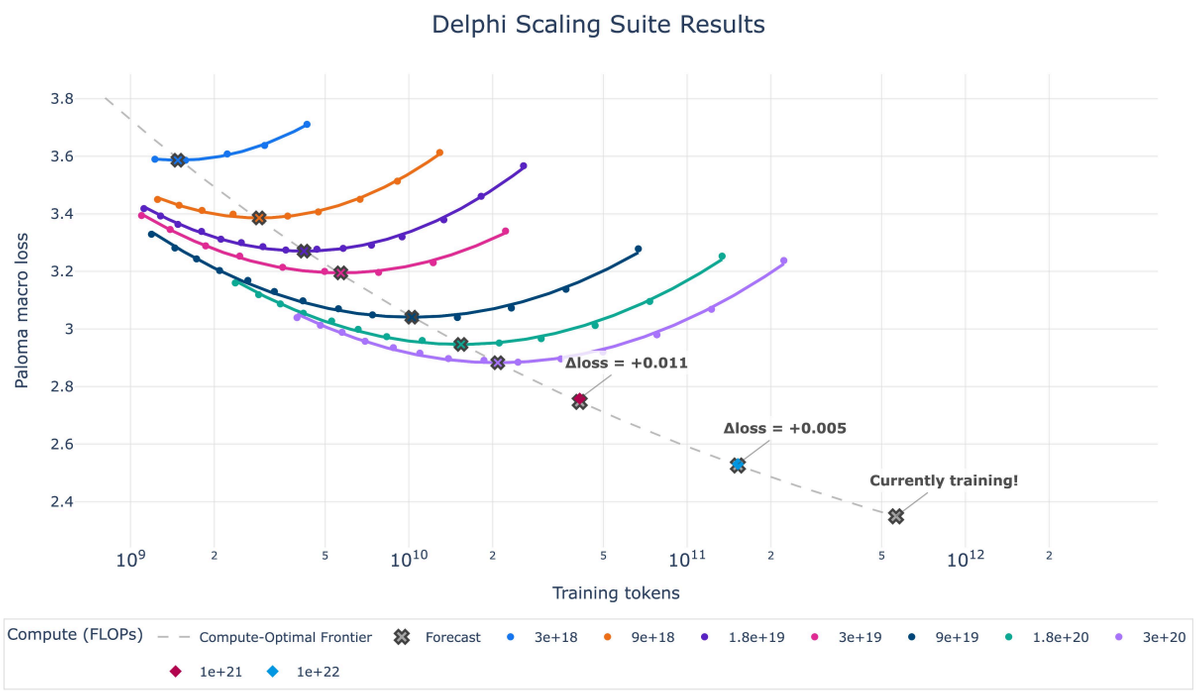
> TL;DR：D = 20 N 大致最优（例如 70B 参数的模型应该在约 1.4T token 上训练）

Caveat: this doesn't take into account inference costs (want a smaller model)

> 注意：这没有考虑推理成本（我们会想要更小的模型）

Live example from Marin [post]

> 来自 Marin 的现场示例 [post]



Should be done training this week, should see how well we match the preregistered loss!

> 本周应该能训练完，到时候看看我们与预注册损失的吻合程度！

Assignment 3 (scaling laws)

> 作业 3（scaling laws）

[GitHub][PDF from Spring 2025]

- We define a training API (hyperparameters → loss) based on previous runs

> 我们基于以往的运行定义一个训练 API（超参数 → 损失）

- Submit "training jobs" (under a FLOPs budget) and gather data points

> 提交"训练作业"（在 FLOPs 预算内）并收集数据点

- Fit scaling laws to the data points

> 对数据点拟合 scaling laws

- Submit extrapolated hyperparameters and loss predictions

> 提交外推得到的超参数和损失预测

- Leaderboard: minimize loss given FLOPs budget

> 排行榜：在给定 FLOPs 预算下最小化损失

Question: What capabilities do we want the model to have?

> 问题：我们希望模型具备哪些能力?

Multilingual? Good at conversation? Agentic coding capabilities?

> 多语言？擅长对话？智能体编程能力?

Evaluation

> 评测

What is the purpose of evaluation?

> 评测的目的是什么?

1. Internal: guide model development (smoothness across scales, relative performance matters)

> 内部：指导模型研发（跨规模的平滑性、相对性能更重要）

2. External: measure absolute quality of a real use case (ecological validity matters)

> 外部：衡量真实应用场景的绝对质量（生态效度很重要）

Examples of evaluations:

> 评测的例子：

1. Perplexity: ideally run on private documents not on Internet (avoid contamination)

> 困惑度：最好在未公开的私有文档上评估（避免污染）

2. Advanced use cases: GPQA, HLE, SWE-Bench, Terminal-Bench

> 高级应用场景：GPQA、HLE、SWE-Bench、Terminal-Bench

LMs are general purpose, require a diverse set of evaluations!

> LM 是通用的，需要一套多样化的评测！

Data curation

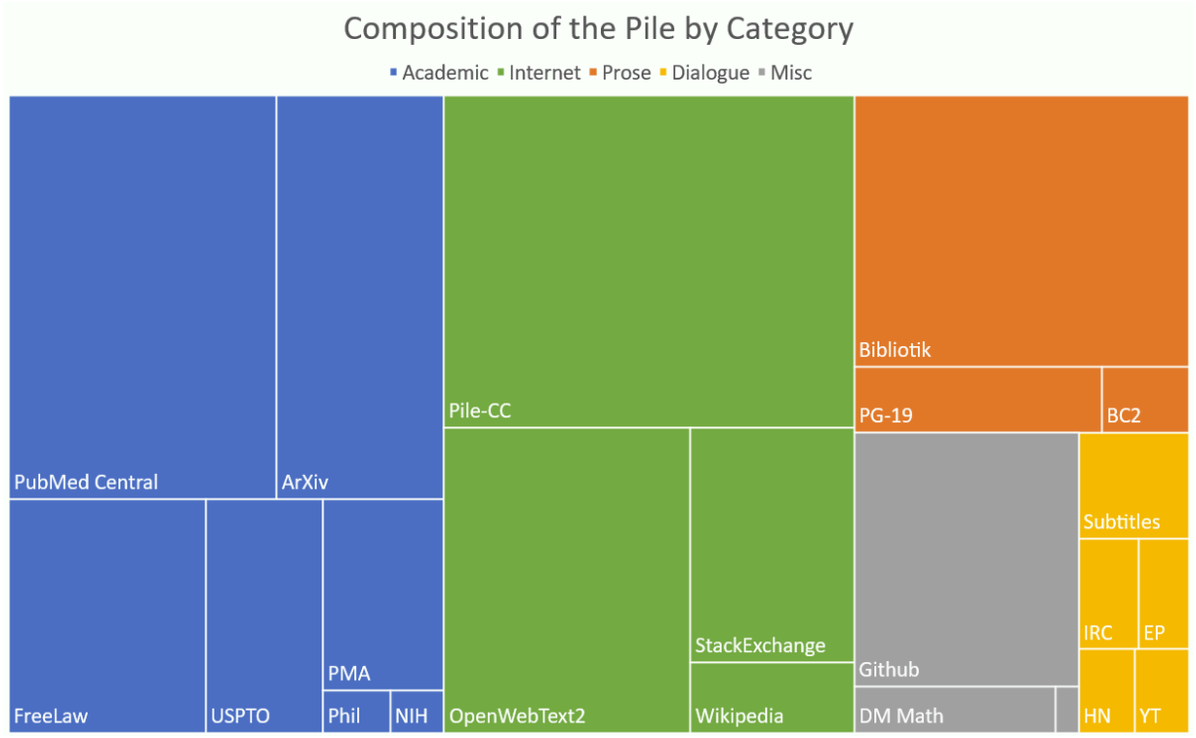
> 数据治理

- Data does not just fall from the sky.

> 数据不会从天而降。

- Sources: webpages crawled from the Internet, books, arXiv papers, GitHub code, etc.

> 来源：从互联网爬取的网页、书籍、arXiv 论文、GitHub 代码等



- Appeal to fair use to train on copyright data? [Henderson+ 2023]

> 以合理使用为由在版权数据上训练？ [Henderson+ 2023]

- Might have to license data (e.g., Google with Reddit data) [article]

> 可能不得不为数据取得授权（例如 Google 与 Reddit 的数据） [article]

- Raw data is HTML, PDF, directories (not text), requires processing

> 原始数据是 HTML、PDF、目录（不是文本），需要处理

Data processing

> 数据处理

- Transformation: convert HTML/PDF to text (extract main content)

> 转换：把 HTML/PDF 转成文本（提取正文）

- Filtering: keep high quality data, remove harmful content (via classifiers)

> 过滤：保留高质量数据，移除有害内容（通过分类器）

- Deduplication: save compute, avoid memorization; use Bloom filters or MinHash

> 去重：节省算力、避免记忆化；使用 Bloom filter 或 MinHash

- Data mixing: how much to upweight/downweight each source? [Liu+ 2024][Chen+ 2026]

> 数据配比：每个来源该上调/下调多少权重？ [Liu+ 2024][Chen+ 2026]

- Rewriting / synthetic data: use LM to augment real data, more similar to downstream tasks [Maini+ 2024]

> 改写/合成数据：用 LM 增强真实数据，使其更贴近下游任务 [Maini+ 2024]

Types of data:

> 数据的类型：

- Pretraining data: large and diverse

> 预训练数据：量大且多样

- Mid-training data: high quality, including long-context

> 中期训练数据：高质量，包含长上下文

- Post-training data: supervised fine-tuning (conversations, agentic traces with tool calling)

> 后训练数据：有监督微调（对话、带工具调用的智能体轨迹）

Assignment 4 (data)

> 作业 4（数据）

[GitHub][PDF from Spring 2025]

- Convert Common Crawl HTML to text

> 把 Common Crawl 的 HTML 转成文本

- Train classifiers to filter for quality and harmful content

> 训练分类器，按质量和有害内容进行过滤

- Deduplication using MinHash

> 用 MinHash 去重

- Leaderboard: minimize perplexity given token budget

> 排行榜：在给定 token 预算下最小化困惑度

So far, we have trained a model on full supervision (predict the next token).

> 到目前为止，我们是在完全监督下训练模型（预测下一个 token）。

Now that the model should be reasonable, we can improve it further from **weak supervision**.

> 如今模型应该已经像样了，我们可以借助弱监督进一步提升它。

Why weak supervision? When it is easier to critique than to generate.

> 为什么要弱监督？当"评价"比"生成"更容易的时候。

Basic template:

> 基本模板：

1. Generate responses from the model.

> 让模型生成回答。

2. Score responses with a {human, verifier, LM judge}.

> 用{人类、验证器、LM 裁判}给回答打分。

3. Update the model to prefer better responses.

> 更新模型，使其偏好更好的回答。

Algorithms:

> 算法：

- Proximal Policy Optimization (PPO) from reinforcement learning [Schulman+ 2017][Ouyang+ 2022]

> 源自强化学习的近端策略优化（PPO） [Schulman+ 2017][Ouyang+ 2022]

- Direct Policy Optimization (DPO): for preference data, simpler [Rafailov+ 2023]

> 直接策略优化（DPO）：面向偏好数据，更简单 [Rafailov+ 2023]

- Group Relative Preference Optimization (GRPO): remove value function [Shao+ 2024]

> 组相对偏好优化（GRPO）：去掉价值函数 [Shao+ 2024]

Challenges:

> 挑战：

- RL algorithms are unstable and hard to tune

> RL 算法不稳定且难以调参

- At scale, this requires a lot of new infrastructure (inference with async rollouts)

> 规模化时需要大量新的基础设施（带异步 rollout 的推理）

- Constantly trading off systems efficiency and on-policyness

> 不断在系统效率与 on-policy 程度之间做取舍

Assignment 5 (alignment)

> 作业 5（对齐）

[GitHub][PDF from Spring 2025]

- Implement Direct Preference Optimization (DPO)

> 实现直接偏好优化（DPO）

- Implement Group Relative Preference Optimization (GRPO)

> 实现组相对偏好优化（GRPO）

This unit was inspired by Andrej Karpathy's video on tokenization; check it out! [video]

> 本单元的灵感来自 Andrej Karpathy 关于分词的视频，值得一看！ [video]

Summary:

> 总结：

- Tokenizer: strings ↔ tokens (indices)

> Tokenizer：字符串 ↔ token（索引）

- Character-based, byte-based, word-based tokenization are highly suboptimal

> 基于字符、基于字节、基于词的分词都远非最优

- BPE is an effective heuristic that is data-driven

> BPE 是一种有效的、数据驱动的启发式方法

- Tokenization is a separate step, maybe one day do it end-to-end from bytes...

> 分词是一个独立步骤，也许有一天能直接从字节端到端完成……

But whatever solution needs to satisfy:

> 但无论哪种方案，都需要满足：

1. Model (e.g., Transformer) should operate on chunks (abstractions) of the sequence (text, video, DNA, etc.)

> 模型（如 Transformer）应该在序列（文本、视频、DNA 等）的块（抽象）上运算

2. Chunks should be variable (allocate more model capacity to interesting chunks)

> 块应当可变（把更多模型容量分配给有趣的块）

Raw text is generally represented as Unicode strings.

> 原始文本通常表示为 Unicode 字符串。

A language model places a probability distribution over sequences of tokens (usually represented by integer indices).

> 语言模型在 token 序列（通常用整数索引表示）上定义概率分布。

So we need a procedure that *encodes* strings into tokens.

> 所以我们需要一个把字符串编码成 token 的过程。

We also need a procedure that *decodes* tokens back into strings.

> 我们还需要一个把 token 解码回字符串的过程。

A Tokenizer is a class that implements the encode and decode methods.

> Tokenizer 是一个实现了 encode 和 decode 方法的类。

To get a feel for how tokenizers work, play with this [interactive site]

> 想直观感受分词器的工作方式，可以玩玩这个 [interactive site]

Observations

> 观察

- A word and its preceding space are part of the same token (e.g., " world").

> 一个词和它前面的空格属于同一个 token（例如 " world"）。

- A word at the beginning and in the middle are represented differently (e.g., "hello hello").

> 出现在开头与出现在中间的同一个词，表示形式不同（例如 "hello hello"）。

- Numbers are tokenized into every few digits.

> 数字会被按每几位数字切成 token。

Here's the GPT-5 tokenizer from OpenAI (tiktoken) in action.

> 这是 OpenAI 的 GPT-5 分词器（tiktoken）的实际运行效果。

Check that encode() and decode() roundtrip:

> 验证 encode() 和 decode() 能往返转换：

Compression ratio: number of bytes per token

> 压缩比：每个 token 对应的字节数

The larger the compression ratio, the shorter the sequence (good since attention is quadratic in sequence length).

> 压缩比越大，序列越短（这是好事，因为注意力关于序列长度是二次的）。

One could increase compression ratio by increasing **vocabulary size** (number of possible token values increases), leading to sparsity.

> 可以通过增大词表来提高压缩比（可能的 token 取值数变多），但这会带来稀疏性。

Let's take a look at the actual vocabulary: [vocab]

> 来看看实际的词表： [vocab]

A Unicode string is a sequence of Unicode characters.

> Unicode 字符串是 Unicode 字符的序列。

Each character can be converted into a code point (integer) via `ord`.

> 每个字符都可以通过 ord 转换为码点（整数）。

It can be converted back via `chr`.

> 可以通过 chr 转换回去。

Now let's build a `Tokenizer` and make sure it round-trips:

> 现在让我们构建一个 Tokenizer 并确认它能往返转换：

There are approximately 150K Unicode characters. [Wikipedia]

> Unicode 字符大约有 150K 个。 [Wikipedia]

Problem 1: this is a very large vocabulary.

> 问题 1：词表非常大。

Problem 2: many characters are quite rare (e.g., 🌍), which is inefficient use of the vocabulary.

> 问题 2：许多字符相当罕见（例如 🌍），这是对词表的低效使用。

This tokenizer is the worst of both worlds (large vocabulary, low compression ratio).

> 这个分词器两头都不占优（词表大、压缩比低）。

Unicode strings can be represented as a sequence of bytes, which can be represented by integers between 0 and 255.

> Unicode 字符串可以表示为字节序列，每个字节可以用 0 到 255 之间的整数表示。

The most common Unicode encoding is [UTF-8]

> 最常见的 Unicode 编码是 [UTF-8]

Some Unicode characters are represented by one byte:

> 有些 Unicode 字符用一个字节表示：

Others take multiple bytes:

> 另一些需要多个字节：

Now let's build a `Tokenizer` and make sure it round-trips:

> 现在让我们构建一个 Tokenizer 并确认它能往返转换：

The vocabulary is nice and small: a byte can represent 256 values.

> 词表非常小：一个字节可以表示 256 个值。

What about the compression rate?

> 那压缩率怎么样?

The compression ratio is terrible, which means the sequences will be too long.

> 压缩比很糟糕，这意味着序列会太长。

Given that the context length of a Transformer is limited (since attention is quadratic), this is not looking great...

> 鉴于 Transformer 的上下文长度有限（因为注意力是二次的），这可不太妙……

Another approach (closer to what was done classically in NLP) is to split strings into words.

> 另一种方法（更接近传统 NLP 的做法）是把字符串切分成词。

This regular expression keeps all alphanumeric characters together (words).

> 这个正则表达式把所有字母数字字符保持在一起（即词）。

To turn this into a `Tokenizer`, we need to map these chunks into integers.

> 要把它变成 Tokenizer，我们需要把这些块映射成整数。

Then, we can build a mapping from each chunk into an integer.

> 然后，我们可以建立从每个块到整数的映射。

What's good: each token is meaningful (since humans invented words).

> 好处：每个 token 都有意义（因为词是人类发明的）。

Compression ratio is good, but vocabulary size can be huge.

> 压缩比不错，但词表可能非常庞大。

Moreover:

> 此外：

- Many words are rare and the model won't learn much about them.

> 许多词很罕见，模型学不到多少关于它们的信息。

- This doesn't obviously provide a fixed vocabulary size.

> 这并不能天然给出固定的词表大小。

- New words we haven't seen during training get a special UNK token, which is ugly and can mess up perplexity calculations.

> 训练时没见过的新词会被赋以特殊的 UNK token，这很难看，还可能搞乱困惑度的计算。

Byte Pair Encoding (BPE)

> 字节对编码（BPE）

The BPE algorithm was introduced by Philip Gage in 1994 for data compression. [article]

> BPE 算法由 Philip Gage 于 1994 年为数据压缩而提出。 [article]

It was adapted to NLP for neural machine translation. [Sennrich+ 2015]

> 它后来被引入 NLP，用于神经机器翻译。 [Sennrich+ 2015]

(Previously, papers had been using word-based tokenization.)

> （在那之前，论文一直使用基于词的分词。）

BPE was then used by GPT-2. [Radford+ 2019]

> BPE 随后被 GPT-2 采用。 [Radford+ 2019]

Basic idea: *train* the tokenizer on raw text to construct a vocabulary tailored to the data.

> 基本思想：在原始文本上训练分词器，构建贴合数据的词表。

Intuition: common sequences of bytes are represented by a single token, rare sequences are represented by many tokens.

> 直觉：常见的字节序列用一个 token 表示，罕见的序列用多个 token 表示。

Sketch: start with each byte as a token, and successively merge the most common pair of adjacent tokens.

> 概要：从每个字节作为一个 token 开始，不断合并最常出现的相邻 token 对。

Training the tokenizer

> 训练分词器

Using the tokenizer

> 使用分词器

Now, given a new text, we can encode it.

> 现在，给定一段新文本，我们可以对它进行编码。

In Assignment 1, you will go beyond this in the following ways:

> 在作业 1 中，你将在以下几个方面更进一步：

- encode() currently loops over all merges. Only loop over merges that matter.

> encode() 目前会遍历所有合并。请只遍历真正相关的合并。

- Detect and preserve special tokens (e.g., <|endoftext|>).

> 检测并保留特殊 token（例如 <|endoftext|>）。

- Use pre-tokenization (e.g., the GPT-2 tokenizer regex).

> 使用预切分（例如 GPT-2 分词器的正则表达式）。

- Try to make the implementation as fast as possible.

> 尽量让实现跑得尽可能快。

Start with the list of bytes of `string`.

> 从字符串的字节列表开始。